

Travel Itinerary Planner - Project Documentation

Table of Contents

1. [Project Overview](#)
 2. [Features](#)
 3. [Technology Stack](#)
 4. [Application Architecture](#)
 5. [How I Created the App](#)
 6. [Key Components & Code Structure](#)
 7. [User Interface & Design](#)
 8. [Data & Functionality](#)
 9. [Screenshots](#)
 10. [Future Enhancements](#)
-

Project Overview

TravelPlanner is a modern, interactive web application built with Next.js that helps users create personalized travel itineraries based on their budget, destination, and trip duration. The app provides:

- **Smart Budget Planning:** Automatically distributes budget across flights, hotels, and activities
- **Real Hotel Recommendations:** Features authentic luxury hotels from around the world
- **Authentic Landmarks:** Includes real tourist attractions and points of interest
- **Multi-Currency Support:** Supports 8 major currencies (USD, EUR, GBP, INR, SEK, JPY, AUD, CAD)
- **Interactive Maps Integration:** Click-to-view locations on Google Maps
- **Beautiful Animations:** Smooth transitions and engaging visual effects

Live Demo: The application runs on localhost:3000 after installation

Features

1. Welcome Page

- Animated landing page with floating clouds and flying planes
- Rotating globe animation
- Feature highlights with hover effects
- Clear call-to-action button

2. Trip Planning Form

- **Origin Selection:** Choose from 23 pre-loaded major cities
- **Destination Selection:** Dynamic list excluding origin city
- **Trip Duration:** 1-30 days
- **Budget Input:** Custom budget amount
- **Currency Selection:** 8 supported currencies with symbols
- Real-time budget display with currency conversion

3. Personalized Itinerary

- **Flight Details:**
 - Realistic flight times based on actual flight durations
 - Departure and arrival times
 - Estimated cost breakdown
- **Hotel Recommendations:**

- Real luxury hotels (e.g., The Plaza Hotel, Burj Al Arab, The Ritz London)
- 5-star ratings
- Check-in/check-out times
- Nightly and total stay costs
- **Daily Activities:**
 - 4-5 activities per day
 - Real landmarks and attractions
 - Time slots for each activity
 - Individual cost estimates
 - Click-to-view on Google Maps
 - Extended time intervals for longer stays with limited attractions
- **Budget Summary:**
 - Total budget
 - Estimated total cost
 - Remaining budget

4. Export & Print

- Print-friendly itinerary layout
 - Plan multiple trips
-

Technology Stack

Core Technologies

- **Framework:** Next.js 16.0.1 (React 19.2.0)
- **Language:** TypeScript
- **Styling:** Tailwind CSS 4.0
- **Build Tool:** Next.js built-in compiler
- **Font Optimization:** Next.js Font System (Geist font)

Development Tools

- **Package Manager:** npm
- **TypeScript Compiler:** TypeScript 5.x
- **React Compiler:** babel-plugin-react-compiler 1.0.0
- **PostCSS:** Tailwind PostCSS 4.0

Key Features Used

- React 19 Server Components
 - Client-side state management with useState
 - TypeScript interfaces for type safety
 - Responsive design with Tailwind CSS
 - CSS animations and transitions
-

Application Architecture

File Structure

```
travel-itinerary/
├── src/
│   ├── app/
│   │   ├── layout.tsx      # Root layout component
│   │   ├── page.tsx       # Main application component
│   │   └── globals.css    # Global styles with Tailwind
│   └── public/           # Static assets
├── package.json          # Dependencies
├── tsconfig.json         # TypeScript configuration
├── next.config.ts        # Next.js configuration
├── tailwind.config.ts    # Tailwind CSS configuration
└── postcss.config.mjs    # PostCSS configuration
```

Component Architecture

The application follows a **single-page application (SPA)** pattern with three main views:

1. **Welcome Screen** (!showForm && !itinerary)
2. **Planning Form** (showForm && !itinerary)
3. **Itinerary Display** (itinerary)

State Management

```
const [showForm, setShowForm] = useState(false);
const [travelDetails, setTravelDetails] = useState<TravelDetails>
({...});
const [itinerary, setItinerary] = useState<Itinerary | null>(null);
```

Data Flow

1. User clicks "Start Planning Your Trip" → setShowForm(true)
 2. User fills form and submits → generateItinerary() → setItinerary(...)
 3. User can reset → resetForm() → Clear all state
-

How I Created the App

Step 1: Project Initialization

I started by creating a new Next.js project using create-next-app:

```
npx create-next-app@latest travel-itinerary --typescript --tailwind --app
```

Configuration choices: - ✓ TypeScript - ✓ Tailwind CSS - ✓ App Router
- ✓ React 19 - ✓ Geist font optimization

Step 2: Planning the Data Structure

I designed TypeScript interfaces to ensure type safety:

```
interface TravelDetails {
  from: string;
  destination: string;
  days: number;
  budget: number;
  currency: string;
}

interface Itinerary {
  flight: {...};
  hotel: {...};
  places: {...}[];
  summary: {...};
}
```

Step 3: Building the Data Layer

I created comprehensive data sets including:

- **23 major cities** with complete travel data
- **Currency exchange rates** for 8 currencies
- **Real hotels** (90+ luxury hotels worldwide)
- **Flight durations** (500+ city-pair combinations)
- **Destination-specific attractions** (180+ real landmarks)

Example data structure:

```
const destinationHotels = {
  'Paris': [
    { name: 'Hôtel Plaza Athénée', rating: 5 },
    { name: 'Le Meurice', rating: 5 },
    // ... more hotels
  ],

```

```

    // ... other cities
  };

  const destinationPlaces = {
    'Paris': [
      {
        name: 'Eiffel Tower',
        desc: 'Visit the iconic iron tower...',
        cost: 30,
        location: 'Eiffel Tower, Paris, France'
      },
      // ... more places
    ],
  };

```

Step 4: Creating the User Interface

Welcome Page

I designed an engaging landing page with: - Gradient background (bg-gradient-to-br from-blue-50 via-white to-purple-50) - Animated elements: - Floating clouds with animate-float - Flying planes with animate-fly-across - Rotating globe with animate-spin-slow - Fade-in animations for text

Planning Form

Built a comprehensive form with: - Dropdown selects for cities - Number inputs for days and budget - Currency selector - Real-time budget preview - Form validation

Itinerary Display

Created an organized layout with: - Flight details section - Hotel accommodation section - Day-by-day activity breakdown - Budget summary cards - Interactive map links

Step 5: Implementing Core Logic

Currency Conversion

```

=> {
  const convertCurrency = (amount: number, currency: string): number
    return Math.round(amount * currencyRates[currency]);
};

```

Flight Time Calculation

```

const calculateFlightTimes = (from: string, to: string) => {
  const duration = flightDurations[from]?.[to] || 8;
  // Calculate realistic departure and arrival times
  // ...
};

```

Itinerary Generation

```

const generateItinerary = (e: React.FormEvent) => {
  // Calculate budget allocation (30% flight, 40% hotel, 30%
  activities)
  // Select random hotel from destination
  // Generate day-by-day activities
  // Calculate totals and remaining budget
};

```

Step 6: Adding Animations & Styling

Created custom animations in globals.css:

```

@keyframes float {
  0%, 100% { transform: translateY(0px); }
  50% { transform: translateY(-20px); }
}

```

```

    }

    @keyframes fly-across {
      0% { left: -100px; }
      100% { left: 100%; }
    }
  }

```

Applied Tailwind classes for: - Responsive layouts (md:grid-cols-2, md:grid-cols-3) - Hover effects (hover:scale-105, hover:shadow-lg) - Transitions (transition-all duration-200) - Gradients (bg-gradient-to-r from-blue-600 to-purple-600)

Step 7: Adding Interactivity

Google Maps Integration

```

const openInMaps = (location: string, placeName: string) => {
  const mapsUrl = `https://www.google.com/maps/search/?api=1&query=${encodeURIComponent(location)}`;
  window.open(mapsUrl, '_blank');
};

```

Print Functionality

```
onClick={() => window.print()}
```

Step 8: Optimization & Polish

- Added loading states
- Implemented form validation
- Created responsive design for mobile
- Optimized animations for performance
- Added helpful tips for longer trips
- Implemented extended time slots for destinations with limited attractions

Step 9: Bug Fixes & Enhancements

Issues addressed: 1. **Repetitive places:** Implemented modulo-based rotation through available attractions 2. **Flight animations on itinerary page:** Removed animations from itinerary view 3.

Extended time intervals: Added time ranges (e.g., "09:00 AM - 12:00 PM") for longer stays with limited places

Key Components & Code Structure

Main Component Structure

```

export default function Home() {
  // State management
  const [showForm, setShowForm] = useState(false);
  const [travelDetails, setTravelDetails] = useState<TravelDetails>(
    {...}
  );
  const [itinerary, setItinerary] = useState<Itinerary | null>(
    null
  );

  // Helper functions
  const convertCurrency = (amount: number, currency: string) =>
    {...};
  const formatCurrency = (amount: number, currency: string) =>
    {...};

  const calculateFlightTimes = (from: string, to: string) => {...};
  const generateItinerary = (e: React.FormEvent) => {...};
  const openInMaps = (location: string, placeName: string) => {...};
  const resetForm = () => {...};

  // Render logic
  return (
    <div>

```

```
        { /* Welcome Section */ }
        { !showForm && !itinerary && <WelcomeView /> }

        { /* Form Section */ }
        { showForm && !itinerary && <FormView /> }

        { /* Itinerary Section */ }
        { itinerary && <ItineraryView /> }
    </div>
  );
}
```

Data Management

All data is stored in-memory as TypeScript objects: - currencyRates: Exchange rates for 8 currencies - currencySymbols: Currency symbols - availableCities: 23 supported cities - destinationHotels: 90+ real hotels - flightDurations: 500+ flight duration mappings - destinationPlaces: 180+ real landmarks and attractions

Styling Approach

Tailwind CSS Utility-First Design: - Responsive breakpoints: md:, lg: - Spacing system: p-8, mb-6, gap-4 - Color palette: blue-600, purple-600, gray-800 - Effects: shadow-lg, hover:shadow-xl, transition-all

Custom CSS: - Animations defined in globals.css - Applied via Tailwind classes: animate-float, animate-fly-across

User Interface & Design

Color Scheme

- **Primary:** Blue gradient (from-blue-600 to-purple-600)
- **Background:** Light gradient (from-blue-50 via-white to-purple-50)
- **Accents:** Green for costs, Purple for highlights
- **Text:** Gray scale for hierarchy

Typography

- **Font:** Geist (optimized by Next.js)
- **Headings:** Bold, large sizes (2xl-6xl)
- **Body:** Regular weight, readable sizes

Layout Principles

- **Centered content** with max-width containers
- **Card-based design** with rounded corners and shadows
- **Grid layouts** for organizing information
- **Responsive design** adapting to all screen sizes

Animation Strategy

- **Subtle animations** on welcome page (floating, spinning)
 - **No animations** on itinerary page (per requirement)
 - **Hover effects** for interactivity
 - **Smooth transitions** for state changes
-

Data & Functionality

Budget Allocation Algorithm

```
const flightCost = Math.min(travelDetails.budget * 0.3, 500);
const hotelNightlyRate = Math.min((travelDetails.budget * 0.4) /
```

```
travelDetails.days, 200);
    const hotelTotalCost = hotelNightlyRate * travelDetails.days;
    const dailyActivityBudget = (travelDetails.budget - flightCost -
hotelTotalCost) / travelDetails.days;
```

Breakdown: - **30%** for flights (capped at \$500) - **40%** for hotels - **30%** for activities and experiences

Activity Distribution Logic

```
    const totalPlaces = placesData.length;
    const activitiesPerDay = totalPlaces >= 5 ? 5 :
Math.min(totalPlaces, 4);
    const useExtendedTimeSlots = (totalPlaces >= 6 && totalPlaces <= 7
&& travelDetails.days >= 5);

    // Rotate through places to avoid repetition
    const placeIndex = (dayIndex * adjustedActivitiesPerDay + i) %
placesData.length;
```

Flight Time Calculation

Based on real flight durations between city pairs: - Departure time: Random between 8 AM - 2 PM - Arrival time: Calculated using actual flight duration - Format: 12-hour time with AM/PM

Screenshots

1. Welcome Page

Description: The landing page features a beautiful gradient background with animated clouds and flying planes. A large rotating globe sits at the center with the "TravelPlanner" title in gradient text. Three feature cards highlight Smart Planning, Real Hotel Recommendations, and Real Landmarks.

[Screenshot would show the welcome page with animations]

2. Planning Form

Description: A clean, centered form with dropdown selects for origin and destination cities, number inputs for days and budget, and a currency selector. The form includes helpful validation messages and real-time budget preview.

[Screenshot would show the form interface]

3. Flight Details Section

Description: A white card displaying flight information in a grid layout with blue and purple accents. Shows origin, destination, departure time, arrival time, and estimated cost.

[Screenshot would show the flight details card]

4. Hotel Accommodation Section

Description: A detailed hotel card showing the hotel name, star rating, check-in/check-out times, nightly rate, and total stay cost with green accents.

[Screenshot would show the hotel section]

5. Daily Itinerary

Description: Day-by-day breakdown with blue vertical line indicators. Each activity shows time, location name, description, and cost. Activities are clickable to view on Google Maps.

[Screenshot would show the daily itinerary section]

6. Trip Summary

Description: A gradient card (blue to purple) displaying three key metrics: Total Budget, Estimated Cost, and Remaining Budget in large, bold numbers.

[Screenshot would show the summary section]

7. Mobile Responsive View

Description: The application adapts beautifully to mobile devices with stacked layouts, full-width cards, and touch-friendly buttons.

[Screenshot would show mobile view]

8. Example Itinerary - Paris

Description: A complete itinerary for a 5-day trip to Paris showing flights from New York, stay at Hôtel Plaza Athénée, and daily visits to Eiffel Tower, Louvre Museum, Notre-Dame, Arc de Triomphe, and more.

[Screenshot would show a complete Paris itinerary]

Future Enhancements

Planned Features

1. **User Authentication:** Save and retrieve past itineraries
2. **Real-time Flight Prices:** Integration with flight APIs
3. **Weather Information:** Display weather forecasts for destination
4. **Collaborative Planning:** Share itineraries with travel companions
5. **Booking Integration:** Direct links to hotel and flight booking
6. **Custom Activities:** Allow users to add their own activities
7. **Export to PDF:** Generate downloadable PDF itineraries
8. **Social Sharing:** Share trips on social media
9. **Reviews & Ratings:** Add user reviews for destinations
10. **Multi-city Trips:** Support for multiple destinations

Technical Improvements

1. **Backend Integration:** Connect to database for persistence
 2. **API Development:** Create RESTful API for data access
 3. **Testing:** Add unit and integration tests
 4. **Performance:** Implement code splitting and lazy loading
 5. **Accessibility:** Enhance ARIA labels and keyboard navigation
 6. **Internationalization:** Support multiple languages
 7. **PWA:** Convert to Progressive Web App for offline access
-

Conclusion

This Travel Itinerary Planner demonstrates modern web development practices using Next.js, React 19, and TypeScript. The app combines beautiful design with practical functionality to help users plan their perfect trips.

Key Achievements: - ✓ Clean, maintainable code with TypeScript - ✓ Beautiful, responsive UI with Tailwind CSS - ✓ Rich data set with 23 cities and 180+ landmarks - ✓ Smart budget allocation algorithm - ✓ Interactive maps integration - ✓ Smooth animations and transitions - ✓ Multi-currency support

Technologies Mastered: - Next.js 16 App Router - React 19 with Hooks - TypeScript for type safety - Tailwind CSS for styling - CSS animations - Responsive design principles

Getting Started

Installation

```
# Clone the repository
git clone [repository-url]

# Navigate to project
cd travel-itenary

# Install dependencies
npm install

# Run development server
npm run dev
```

Access the App

Open <http://localhost:3000> in your browser.

Build for Production

```
npm run build
npm start
```

Author

Created with ❤️ as a modern web development project showcasing Next.js, React 19, and TypeScript capabilities.

Repository: [GitHub Link] **Live Demo:** [Deployment Link]

Last Updated: December 2025